
Building Multi-Tenant SaaS with Go and PostgreSQL

A practical guide to schema-per-tenant isolation in production systems

1 Introduction

Multi-tenancy is a **fundamental requirement** for any modern SaaS platform. The challenge lies in choosing the right isolation strategy that balances **security**, **performance**, and **operational complexity**.

In this article, we explore the schema-per-tenant approach using PostgreSQL and Go, drawing from real production experience with a B2B platform serving 200+ tenants.

2 Isolation Strategies

There are three common approaches to multi-tenant data isolation:

Strategy	Isolation Level	Complexity	Cost
Shared tables	Low (row-level)	Low	Low
Schema-per-tenant	Medium (schema-level)	Medium	Medium
Database-per-tenant	High (full isolation)	High	High

3 Why Schema-Per-Tenant

The schema-per-tenant approach offers the best balance for most SaaS applications:

- Strong isolation without the overhead of separate databases
- Easy per-tenant migrations using `search_path`
- Shared connection pooling across all tenants
- Simple backup and restore per tenant
- Compatible with PostgreSQL's built-in tooling

4 Implementation in Go

The key to schema-per-tenant in Go is middleware that sets the PostgreSQL `search_path` on each request:

```
func TenantMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        tenantID := r.Header.Get("X-Tenant-ID")
        if tenantID == "" {
            http.Error(w, "missing tenant", 400)
            return
        }

        schema := "tenant_" + tenantID
        ctx := context.WithValue(r.Context(), "schema", schema)
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}
```

5 Migration Management

Each tenant schema needs independent migrations. We use `goose` with a custom driver:

1. Detect all tenant schemas from the `tenants` table
2. For each schema, set `search_path` and run pending migrations
3. Track migration state per-schema in each tenant's `goose_db_version` table
4. Support rollback per-tenant without affecting others

Always test migrations against a snapshot of production data. Schema-level migrations can still fail on tenant-specific data patterns that don't exist in your dev environment.

6 Performance Considerations

At 200+ tenants, we've observed these key performance patterns:

- **Connection pooling** is critical — use `pgbouncer` with transaction-level pooling
- **Schema count** doesn't significantly impact query performance
- **Index bloat** is the real enemy — schedule regular `REINDEX` operations
- **Monitoring** per-tenant query times helps identify problem tenants early

7 Conclusion

Schema-per-tenant isolation with PostgreSQL and Go provides a pragmatic path to multi-tenancy that scales well into the hundreds of tenants. The approach leverages PostgreSQL's native schema support, keeps operational complexity manageable, and provides strong enough isolation for most B2B SaaS use cases.

Published March 2026. Based on real-world production experience with a multi-tenant B2B platform.